

EE5239 Introduction to Optimization Homework 4 Notes

In this note I provide a few pointers about the homework. Again which algorithm you use for which model is pretty open, here I just want to help set up the problem. Also it would be great if at least you use once the gradient descent algorithm (with different stepsizes) and once the coordinate descent algorithm.

This note is built upon HW 3's notes.

1. Hope that you already got access to the data and can import and plot them as well. If not, please consult the code 'read_plot.m' that I have updated online.
2. Now let's look at the models. Suppose we want to do a linear regression on this problem, how do we set up the model?

Features: We have two features, 'intensity' and 'symmetry', and about 7291 training samples. So the data matrix $\tilde{\mathbf{A}}$ is of size 7291×2 . Our decision variable will be $\tilde{\mathbf{x}} = [x_1, x_2]$. Is this enough? Not really. Recall that in Lecture 3, we mentioned that the linear prediction model should be something of the following form (the notation here is a bit different from lecture 3)

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{a} + x_0$$

That is, for any given data point $\mathbf{a}$, the model produces the prediction of the above form. So here x_0 is the variable that characterizes the "intercept" of the line. Therefore in our problem, we need the following classifier

$$x_1 a_1 + x_2 a_2 + x_0 = 0$$

which precisely defines a line on a 2-dimensional plane. Therefore, we need another decision variable, so the final variable vector should be $\mathbf{x} = [x_1, x_2, x_0]^T \in \mathbb{R}^3$, and the final data matrix should be

$$\mathbf{A} = [\tilde{\mathbf{A}}, \mathbf{1}] \in \mathbb{R}^{7291 \times 3}, \text{ with } \mathbf{1} \text{ being the all 1 column vector of size 7291}$$

Now it is clear that once the model is learned (x_0, x_1, x_2 's are known), the classifier can be plotted by plotting the following line over the 2-dimensional plane

$$x_1 a_1 + x_2 a_2 + x_0 = 0.$$

Labels: What's the label $\mathbf{b}$? In order to do a linear regression for classification, we need to *setup our own labels*. Suppose I am going to classify digit '2' and the rest. Then for all instances i correspond to the letter 2, I will label $b_i = +1$, and for the rest of the instances, I will label their b_j 's as -1 . In the training stage, the model will be

$$\min \|\mathbf{Ax} - \mathbf{b}\|^2$$

where $\mathbf{A}$ and $\mathbf{b}$ are defined above. The question is how to optimize the above problem to find the global optimal solution? Can you try coordinate descent? Gradient Descent? For gradient descent what should be a good stepsize? Can you compare how fast these algorithms do in terms of reducing the objective? You will see that for gradient descent if you don't pick the stepsize carefully, the algorithm will diverge.

Testing: In the testing stage, when a data point $\mathbf{a}$ is given, we classify by $\text{sgn}(\mathbf{a}^T \mathbf{w})$, i.e., if $\mathbf{a}^T \mathbf{x} > 0$ we say that it is the digit of '2' and otherwise it is the rest of the digits. Compute the testing error for the testing data set (how many percent can the classifier correctly classify?)

3. **Modeling:** The same approach as the above can be done for the logistic regression. There we still have 3-dimensional variables, still need to augment the data matrices, and still need to construct the labels $\mathbf{b}$ by ourselves. After we do all that, we have a logistic regression model to solve.

Training: Again you can use gradient descent method to solve it, and show the progress of the algorithm by plotting the decrease of the objective. One more thing about gradient descent. You can pick the diminishing stepsize, or the constant stepsize. For constant stepsize, think about what is the constant that you should pick in this case?

Hint: The logistic regression solves the following problem (where M is the number of data points)

$$\min L(\mathbf{x}) = \frac{1}{M} \sum_{i=1}^M \ln \left(1 + e^{-b_i \mathbf{x}^T \mathbf{a}_i} \right)$$

The gradient of the objective for logistic regression is

$$\nabla L(\mathbf{x}) = -\frac{1}{M} \sum_{i=1}^M \frac{b_i \mathbf{a}_i}{1 + e^{b_i \mathbf{a}_i^T \mathbf{x}}}$$

So we have

$$\nabla^2 L(\mathbf{x}) = \frac{1}{M} \sum_{i=1}^M \frac{b_i^2 \mathbf{a}_i \mathbf{a}_i^T e^{b_i \mathbf{a}_i^T \mathbf{x}}}{(1 + e^{b_i \mathbf{a}_i^T \mathbf{x}})^2} \quad (1)$$

$$(2)$$

Clearly for any $s \geq 0$ we have

$$\frac{e^s}{(1 + e^s)^2} \leq 1,$$

then (note that $b_i^2 = 1$ because b_i 's are -1 and $+1$)

$$\|\nabla^2 L(\mathbf{x})\| \leq \frac{1}{M} \sum_{i=1}^M \|b_i^2 \mathbf{a}_i \mathbf{a}_i^T\| = \frac{1}{M} \sum_{i=1}^M \|\mathbf{a}_i\|^2.$$

The inverse of this gives you the constant to work with.

Can you do the similar analysis, but applied to the *coordinate descent algorithm* for the logistic regression?

Testing: After we have learned the linear model, for each training data $\mathbf{a}$, we know the *probability* of it being in class ‘+1’ with the probability of

$$p(\mathbf{a}) = \frac{\exp(\mathbf{a}^T \mathbf{x})}{1 + \exp(\mathbf{a}^T \mathbf{x})}.$$

Now a simple procedure is to say that every test point that has probability greater than 0.5 will be classified as ‘+1’.

4. Due to the fact that the real-data is almost never linearly separable, a direct application of the Perceptron algorithm is not going work (see Lecture 4). Then the modified Perceptron method has to be used, which checks the decrease of the objective function $L(\mathbf{x})$ every time

$$L(\mathbf{x}) = \sum_{i=1}^m \max\{-b_i \mathbf{x}^T \mathbf{a}_i, 0\} := \sum_{i=1}^m g_i(\mathbf{x})$$

If between two passes over all the misclassified data points, the algorithm still do not give you any reduction in the objective, then the algorithm can stop (you can also pick other stopping criteria as well). Plot the objective decrease for this algorithm and plot how the classifier looks like visually.